University of Southern Brittany

Internship Project

Side-Channel Analysis of a Hardware Keccak-f[1600] Implementation for Post-Quantum Cryptography

---

Author:

Michela Mei

Project Supervisor:

Lejla Batina

Azade Rezaeezade

Academic Year: 2025-2026

Abstract

This project investigates the side-channel vulnerability of the Keccak-f permutation within a unified hardware architecture for ML-KEM and ML-DSA. While Keccak is algorithmically robust, its physical implementation often leaks sensitive data through power or electromagnetic emissions. By developing a 1600-bit emulator reproducing the first-round behavior of the target VHDL implementation, an exploitable intermediate state located in the non-linear χ layer was identified and selected as the attack target. To exploit this physical leakage, I used an hybrid framework combining Correlation Power Analysis (CPA) for leakage assessment and Point-of-Interest (POI) selection with a 1D Convolutional Neural Network (CNN) configured under a multi-label Flexible Bit Model. The network is optimized through an automated grid-search hyperparameter tuning engine and trained with binary cross-entropy loss to predict the 64 bits of the selected internal state lane directly from the attack traces. Experimental evaluations on a dedicated attack dataset demonstrate an average bit recovery accuracy of 75.59%, with up to 53 correctly recovered bits out of 64 for the best attack trace. Finally, the continuous sigmoid probabilities generated by the CNN are intended to serve as soft information for a future SASCA framework.

Keywords: cybersecurity, Keccak (SHAKE), Post-Quantum Cryptography (PQC), Side-Channel Analysis, Soft Analysis Side Channel Attack (SASCA), Deep Learning, Convolutional Neural Network (CNN), Hamming Distance (HD), Flexible Bit Model (FBM).

Acknowledgements

I would like to thank my internship supervisors for their help and guidance, as well as the UBS professors and Erasmus Mundus Joint Master Program, that made it possible to enjoy this incredible year.
I would finally thank all my family and friends for their support and their love, which overcomes the space, the ocean and the time.

Contents

List of Figures

List of Tables

Acronyms

BCE Binary Cross-Entropy. V

BP Belief Propagation. V

CNN Convolutional Neural Network. V

CPA Correlation Power Analysis. V

DL Deep Learning. V

DPA Differential Power Analysis. V

FBM Flexible Bit Model. V

HD Hamming Distance. V

HW Hamming Weight. V

ML-DSA Module-Lattice-Based Digital Signature Algorithm. V

ML-KEM Module-Lattice-Based Key-Encapsulation Mechanism. V

NIST National Institute of Standards and Technology. V

POI Point of Interest. V

PQC Post Quantum Cryptography. V

SASCA Soft Analytical Side-Channel Attack. V

SCA Side Channel Analysis. V

SHA-3 Secure Hash Algorithm 3. V

SHAKE Secure Hash Algorithm KECCAK Extendable-output. V

SNR Signal-to-Noise Ratio. V

1 Introduction

1.1 Project Environment

This project was executed as part of a research internship in collaboration with the Cryptographic Engineering & Side-Channel Analysis (CESCA) Lab at Redboud University.

The project was supervised by the Professor Lejla Batina and Post-Doc Researcher Azade Rezaeezade, who guided me through its development. The work was performed remotely, with regular interaction through weekly online meetings, technical discussions, and progress reviews.

The study focused on side-channel analysis techniques applied to post-quantum cryptographic implementations. Specifically, the research was centered on the security evaluation of a hardware implementation of the Keccak-f[1600] permutation, which is a key component of several NIST-standardized post-quantum cryptographic schemes.

The project was an opportunity to get real experience in the fields of cryptographic engineering, side-channel evaluation, and applied machine learning for cybersecurity.

The results obtained during the internship constitute the basis of the work presented in this report.

The complete source of the developed code is publicly available in a github repository linked in the Appendix-A.

1.2 Motivation

The increasing utilization of cryptographic algorithms raised concerns regarding their ability to withstand physical attacks. Modern cryptographic algorithms are generally designed to be mathematically secure, however, practical implementations may unintentionally leak sensitive information through observable physical events such as power consumption, electromagnetic radiation, timing variations, or acoustic emissions. These leaks can be exploited using Side-Channel Analysis (SCA) techniques, allowing an attacker to recover secrets without directly breaking the cryptographic algorithm.

In recent years, the emergence of Post-Quantum Cryptography (PQC) has introduced new cryptographic constructions designed to resist attacks from quantum computers. Some of these schemes are significantly based on the Keccak permutation, which is the basis of the SHA-3 hash standard and the SHAKE extendable-output functions. As a consequence, evaluating the side-channel resistance of Keccak-based hardware implementations has become an important research topic.

Unlike traditional cryptanalysis, side-channel attacks target the physical implementation instead of the mathematical design of a cryptographic algorithm. Even if the algorithm itself is secure, information leaked during computations can provide enough information to extrapolate secret-dependent variables. It means that understanding how information propagates through the internal state of a cryptographic implementation is essential to develop both effective attack methodologies and suitable countermeasures.

Among the available side-channel techniques, profiling attacks represent one of the most powerful classes of attacks. This approach uses a profiling device to construct machine-learning models in order to predict variables from measured leakage traces.

Another significant side-channel approach is the Soft Analytical Side-Channel Attack (SASCA). SASCA combines probabilistic information extracted from leakage measurements with algebraic knowledge of the cryptographic algorithm. By modeling the algorithm as a

factor graph and propagating probabilistic beliefs it can recover secret information even when individual leakage sources provide only limited information.

This project focuses on the development of a profiling framework targeting a hardware implementation of the Keccak permutation. The obtained probabilistic information is intended to serve as a foundation for future integration into a SASCA framework.

1.3 Problem Statement

The target of this work is an unprotected hardware implementation of the Keccak-f[1600] permutation written in VHDL, where no dedicated side-channel countermeasures are implemented. Consequently, internal state transitions may generate measurable leakage during the execution of the permutation rounds.

The primary challenge addressed in this project is the extraction of information related to internal Keccak state variables from power consumption traces. More specifically, the objective is to identify a suitable intermediate target within the first permutation round and evaluate if machine-learning techniques can recover information about this target from measured side-channel traces.

The recovered information is not limited to a binary decision about a secret value, the goal is to estimate probability distributions associated with internal state bits. These probabilistic outputs will later be integrated into a SASCA framework, where information from multiple variables can be combined using belief propagation techniques.

1.4 Objectives

The main objectives of this project are summarized as follows:

1. Analyze the architecture of the target hardware implementation of Keccak-f[1600].
2. Reconstruct the internal computations of the first Keccak round through software simulation.
3. Identify suitable intermediate variables for side-channel exploitation.
4. Develop a leakage model reflecting the expected behavior of the hardware implementation.
5. Perform Correlation Power Analysis (CPA) to identify Points of Interest (POIs) within the recorded traces.
6. Design and train a Convolutional Neural Network (CNN) capable of predicting internal Keccak state bits.
7. Evaluate the effectiveness of the proposed profiling attack.
8. Investigate the suitability of the obtained probability estimates for future integration into a Soft Analytical Side-Channel Attack framework.

1.5 Contributions

The contributions of this project span across cryptographic engineering, leakage modeling, and deep learning-based security evaluations. Initially, this work provides a detailed analysis of an iterative hardware implementation of Keccak-f[1600], which is the foundation for developing the software reconstruction of the first-round computation, including the Theta, Rho, Pi, and Chi transformations. Using a Hamming Distance leakage model based on internal state transitions, a Correlation Power Analysis (CPA) was performed to select the most relevant Points of Interest (POIs) from the recorded side-channel traces.

Once selected these features, a Convolutional Neural Network (CNN) profiling framework was implemented, specifically targeting a 64-bit internal state lane after the non-linear Chi transformation. The practical efficacy of this deep learning approach was then demonstrated through an experimental evaluation, proving that meaningful internal state information can be successfully recovered from a single side-channel trace.

2 Cryptographic and Side-Channel Background

This section presents the theoretical background required for understanding the Keccak permutation, side-channel analysis techniques, profiling attacks, and Soft Analytical Side-Channel Attacks.

2.1 Cryptographic Hash Functions

Cryptographic hash functions are fundamental in modern cryptography. A hash function maps an input message of arbitrary length to a fixed-size output called *digest*.

A secure hash function should satisfy the following properties:

- Preimage resistance: For a given h in the output space of the hash function, it is *hard* to find any message x with $H(x) = h$.
- Second preimage resistance: For a given message x_1 it is *hard* to find a second message $x_2 \neq x_1$ with $H(x_1) = H(x_2)$.
- Collision resistance: It is *hard* to find a pair of messages $x_1 \neq x_2$ with $H(x_1) = H(x_2)$.

Where *hard* means that there is no polynomial P so that it can be solved in time $P(n)$ for all functions of the family.

Hash functions are widely used in digital signatures, authentication protocols, integrity verification, and post-quantum cryptographic schemes.

2.2 The Sponge Construction

Unlike traditional hash functions, Keccak is based on the sponge construction. The internal state consists of a fixed-size state of $b = 1600$ bits divided into:

- Bitrate (r)
- Capacity (c)

such that:

$$b = r + c$$

The sponge construction operates in two phases:

1. Absorbing phase: message blocks are XORed into the rate portion of the state and followed by the Keccak permutation.
2. Squeezing phase: after all message blocks have been processed, output blocks are extracted from the state.

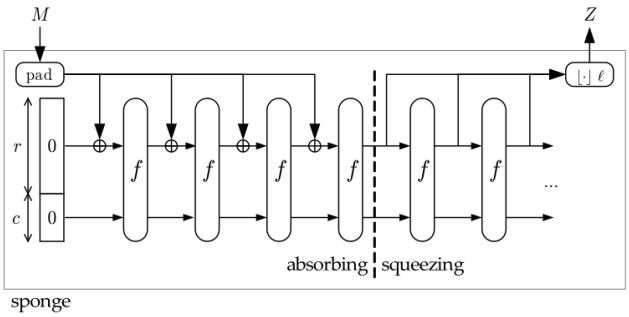

Figure 1: Sponge Construction Keccak, from `https://keccak.team/sponge_duplex.html`

2.3 Keccak-f[1600]

The Keccak-f[1600] permutation operates on a 1600-bit state organized as a matrix of 25 lanes:

$$A[x, y], \quad x, y \in \{0, \ldots, 4\}$$

Each lane contains 64 bits.

A permutation consists of 24 rounds, where each round applies the following transformations:

$$Theta(\theta) \rightarrow Rho(\rho) \rightarrow Pi(\pi) \rightarrow Chi(\chi) \rightarrow Iota(\iota)$$

Theta performs *diffusion* across columns.

Rho and Pi redistribute information through *rotations* and lane *permutations*.

Chi introduces *non-linearity* and is the only non-linear operation in the round function.

Iota injects a round constant into the state.

2.4 Side-Channel Analysis

Physical implementations of cryptographic algorithms may leak information through measurable side channels, such as power consumption.

Power analysis attacks exploit the relationship between processed data and instantaneous power consumption.

2.5 Leakage Models

A leakage model approximates the dependency between internal data and measured power consumption.

Two largely used models are:

- Hamming Weight (HW)
- Hamming Distance (HD)

The Hamming Distance model assumes that power consumption is proportional to the number of bit transitions occurring between two consecutive states:

$$HD(x, y) = HW(x \oplus y)$$

This model is particularly suitable for register-based hardware architectures.

2.6 Profiling Attacks

A profiling phase is used to build a leakage model from known inputs and traces.

The resulting model is then used during the attack phase to extrapolate secret internal variables from previously unseen traces.

Machine learning techniques became increasingly popular in profiling attacks because of their ability to model complex and non-linear leakage behavior.

2.7 Soft Analytical Side-Channel Attacks

Soft Analytical Side-Channel Attacks (SASCA) combine side-channel information with algebraic descriptions of cryptographic algorithms.

Instead of directly predicting secret values, SASCA estimates probability distributions associated with internal variables.

These probabilities are propagated through a factor graph representation of the algorithm using belief propagation techniques.

The combination of probabilistic leakage information and algorithmic constraints often enables attacks that are significantly more powerful than traditional side-channel techniques.

The implementation of a SASCA attack against the Keccak permutation represents the long-term objective of this work.

3 Target Implementation

The goal of this section is to describe the target hardware implementation and to analyze its architecture from a side-channel perspective.

3.1 Description of the Target Implementation

The target of this project is an unprotected hardware implementation of the Keccak-f[1600] permutation written in VHDL. The design supports multiple operating modes, which are selected through a 2-bit control signal.

The implementation follows an iterative design in which one round of the Keccak permutation is executed per clock cycle. A full permutation consists of 24 clock cycles.

The architecture is composed of three main components:

- The internal state register (`reg_s`)
- The round function datapath (Theta, Rho, Pi, Chi, Iota)
- The control logic managing absorption, permutation, and output phases

3.2 Internal State Representation

The Keccak internal state is stored in a 5×5 matrix of 64-bit lanes. In VHDL, this is defined as follows:

```
type k_state is array(0 to 4, 0 to 4)
of std_logic_vector(63 downto 0);

signal reg_s : k_state;
```

The signal `reg_s` represents the current internal state of the permutation. It is updated sequentially at each clock cycle and constitutes the primary storage element of the design.

From a side-channel perspective, transitions of this register are particularly relevant, as they are expected to generate measurable power consumption variations.

The graphical visualization of the state 3D-matrix components is:

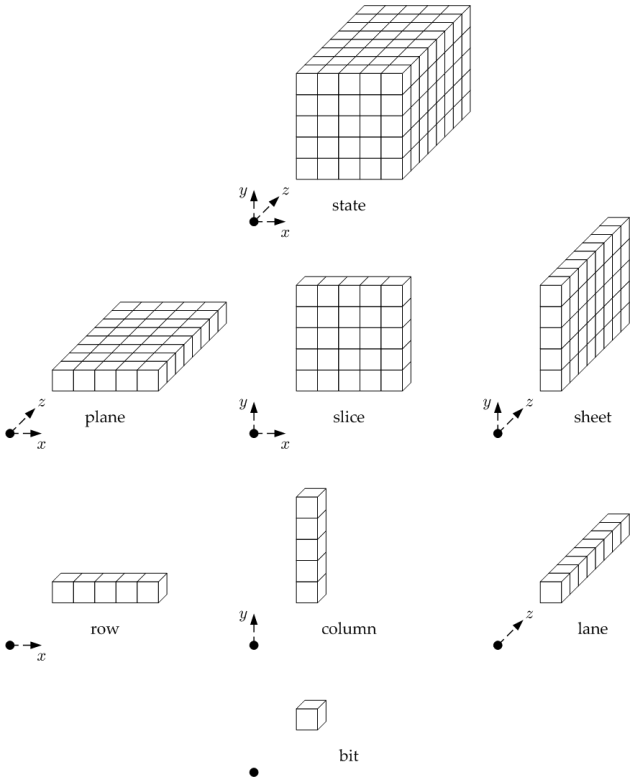

Figure 2: Pieces of Keccak-f state, from `https://keccak.team/figures.html`

3.3 Message Absorption and Mode Selection

Input data is received through a 64-bit streaming interface and absorbed into the rate portion of the Keccak state through XOR operations.

The implementation supports four operating modes:

- SHAKE128
- SHAKE256
- SHA3-256
- SHA3-512

The selected mode determines the bitrate and therefore the number of active lanes involved during the absorption phase.

After the message has been fully absorbed, the standard Keccak padding rule is applied before the permutation process begins.

3.4 Keccak Round Function Implementation

Each Keccak round is composed of five transformations applied sequentially to the internal state:

1. Theta

2. Rho
3. Pi
4. Chi
5. Iota

These transformations are implemented combinationally and the resulting state is written back into the state register at the next clock cycle.

3.4.1 Theta Step

Theta is responsible for the diffusion of information across the state. It computes the parity of each column and propagates this information to neighboring columns through XOR operations and bit rotations.

As a result, every lane becomes dependent on information originating from multiple columns of the state.

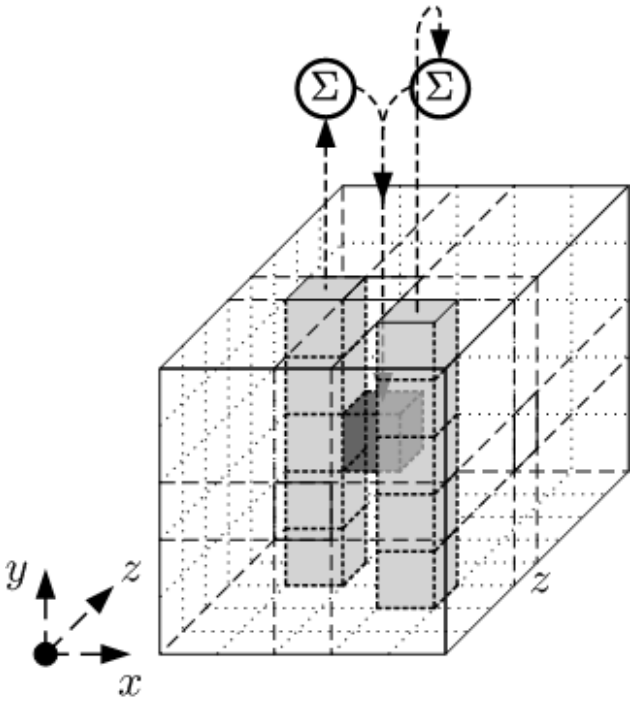

Figure 3: Theta Step, from `https://keccak.team/figures.html`

3.4.2 Rho and Pi Steps

The Rho and Pi transformations are linear operations that redistribute information throughout the state.

Rho applies lane-dependent cyclic rotations.

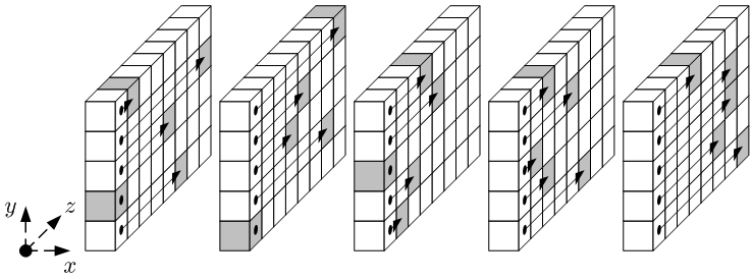

Figure 4: Rho Step, from `https://keccak.team/figures.html`

Pi permutes the coordinates of the lanes according to a fixed mapping.

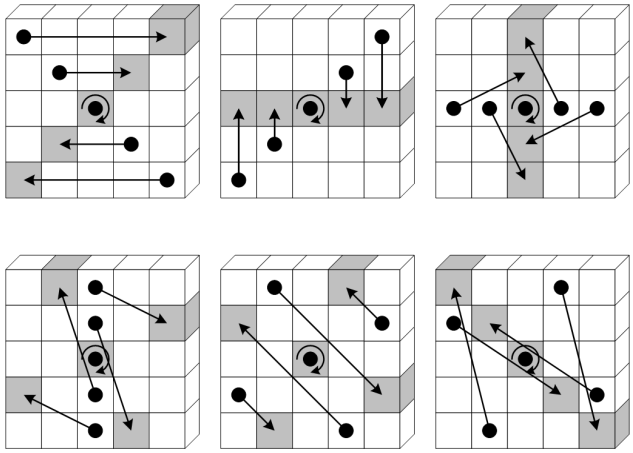

Figure 5: Pi Step, from `https://keccak.team/figures.html`

Together, these operations ensure that local dependencies created during Theta are spread across different regions of the state.

3.4.3 Chi and Iota Steps

The Chi transformation introduces non-linearity into the Keccak permutation and represents the only non-linear step of the round function.

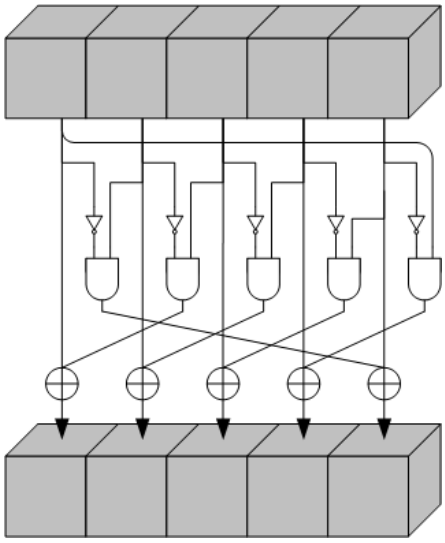

Figure 6: Chi Step, from `https://keccak.team/figures.html`

Its implementation is described by:

```
chi_iota_out(x,y) <= rho_pi_out(x,y) xor
(not rho_pi_out((x+1) mod 5,y) and
rho_pi_out((x+2) mod 5,y));
```

Because of its non-linear nature, Chi plays a fundamental role in the cryptographic strength of Keccak and constitutes the primary target of the side-channel analysis performed in this work.

Finally, the Iota transformation injects a round-dependent constant into a single lane of the state, ensuring asymmetry between rounds.

3.5 Round Execution

The architecture follows an iterative design in which a single Keccak round is executed per clock cycle.

Consequently, the complete Keccak-f[1600] permutation requires 24 consecutive cycles to process all rounds.

This design choice reduces hardware resource utilization while maintaining compliance with the Keccak specification. From a side-channel perspective, the sequential evolution of the state facilitates the observation of intermediate transitions that occur during the round execution.

3.6 Output Extraction

After the completion of all permutation rounds, the internal state is read through the output interface according to the selected operating mode.

This phase corresponds to the squeezing stage of the sponge construction, during which the bitrate portion of the state is sequentially extracted to generate the final hash or extendable output.

3.7 Security Perspective

From a side-channel analysis perspective, the most relevant operation of the architecture is the update of the internal state register:

```
reg_s <= s;
```

This operation is expected to dominate the power consumption profile, since multiple 64-bit lanes may switch simultaneously between two consecutive clock cycles.

For this reason, the leakage behavior of the implementation can be approximated using the Hamming Distance model:

$$HD(x, y) = HW(x \oplus y)$$

which estimates the number of bit transitions occurring between two consecutive states.

Consequently, the evolution of `reg_s` throughout the execution of the permutation represents the primary source of exploitable leakage and forms the basis of the attack methodology presented in the following sections.

4 Experimental Environment

This section describes the experimental setup used to evaluate the side-channel leakage of the target Keccak hardware implementation and to develop the proposed profiling attack methodology. The environment consists of a dataset of power traces, associated cryptographic inputs, and a software framework for preprocessing, modeling, and classification.

The experiments are conducted entirely in a simulated and recorded acquisition environment, where side-channel traces are available in a pre-collected dataset stored in HDF5 format.

4.1 Dataset Description

The dataset used in this work consists of real physical power consumption traces, captured from a hardware implementation and recorded using a digital oscilloscope. The data includes specific metadata detailing the oscilloscope configurations and sampling parameters, which were set to 10 samples per clock cycle.

It contains two main components:

- Power consumption traces
- Corresponding cryptographic inputs

The dataset is structured as follows:

```
traces.shape = (10000, 1980)
inputs.shape = (10000, 8)
```

Each trace corresponds to a single encryption or permutation execution, while each input consists of an 8-byte (64 bits) message block processed by the Keccak core.

The trace length of 1980 samples reflects the full execution of the absorption phase and the Keccak-f permutation rounds.

Two different datasets are created from these data:

	Profiling Set	Attack Set
Traces	[9900,1980]	[100,1980]
Inputs	[9900,8]	[100,8]

Table 1: Datasets

The profiling set is used exclusively to train the model in the profiling phase, while the attack set is only used to perform the attack with the trained model.

4.2 Input Message Reconstruction

The input bytes are combined into 64-bit words in order to reconstruct the message block used in the Keccak state initialization. This reconstruction is performed as follows:

```
msg = np.zeros(len(inputs), dtype=np.uint64)
for i in range(8):
    msg |= inputs[:, i].astype(np.uint64) << (i * 8)
```

This step is necessary to align the dataset representation with the internal format of the Keccak implementation, where each message block is processed as a 64-bit lane.

4.3 Software Environment

The experiments are conducted using a Python-based framework with the following libraries:

- NumPy: numerical computation and array manipulation
- H5Py: loading of HDF5 datasets
- TensorFlow: implementation and training of neural networks
- Scikit-learn: preprocessing, metrics, and data splitting

This combination provides a useful environment for both side-channel analysis techniques and deep learning-based profiling attacks.

4.4 Data Preprocessing

Before applying any analysis technique, the dataset is split into training and testing subsets and trace selection is performed to reduce dimensionality and focus on the most informative regions of the signal.

The preprocessing pipeline includes:

- Traces normalization
- Standardization using zero mean and unit variance
- Selection of Points of Interest (POIs)

Standardization is performed as follows:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

This step ensures that the neural network training is not biased by amplitude differences across traces.

4.5 Target

The attack focuses on intermediate values computed during the first Keccak round, specifically after the Chi transformation. This choice is motivated by the fact that the Chi step introduces non-linearity into the permutation, making it a suitable target for both statistical and machine learning-based attacks.

The selected target is a 64-bit lane of the internal state, which is later used as ground truth for classification in the profiling phase.

5 Leakage Modeling

This chapter describes the leakage model used to analyze the side-channel behavior of the target Keccak implementation. The leakage modeling phase aims to define a mathematical relationship between internal state transitions and observable power consumption.

5.1 Target of the Attack

The profiling attack proposed in this work targets the intermediate state value generated at the output of the non-linear Chi transformation during the first round of the Keccak-f[1600] permutation.

A single 64-bit lane located at coordinates $(x = 3, y = 0)$ is isolated as the target intermediate variable for the side-channel exploitation:

$$A_{target} = A[3, 0]$$

The reason behind selecting this specific lane is related to the diffusion properties of the Keccak round function. Before reaching the Chi layer, the internal state undergoes three consecutive linear transformations: Theta, which diffuses information across neighboring columns, Rho, which performs lane-level cyclic rotations, and Pi, which permutes the lane positions within the matrix.

By the time the data enters the non-linear Chi layer, the bits of $A[3, 0]$ have accumulated a significant dependency on multiple regions of the initial state.

Furthermore, because Chi is the only non-linear transformation in the Keccak round function, it represents a critical point where the algebraic complexity increases.

Targeting $A[3, 0]$ ensures that the physical leakage captured by the side-channel traces correlates with a complex, non-linear function of the input message.

This makes it an ideal intermediate target for both assessing leakage using the CPA and training the CNN.

5.2 Full Round Simulation

In order to compute intermediate values, the Keccak round is fully emulated in software using a faithful implementation of the hardware datapath.

The model includes:

- State initialization
- Theta diffusion
- Rho and Pi permutation
- Chi non-linear layer

It does not include the Iota round constant injections.

This emulation ensures that the intermediate values used for labeling are consistent with the hardware implementation under attack.

5.3 Hamming Distance Leakage Model

The leakage model assumes that power consumption is proportional to the number of bit transitions between two consecutive register states.

Formally, the leakage is defined as:

$$L = HW(S_{old} \oplus S_{new})$$

where:

- S_{old} is the previous state
- S_{new} is the updated state after Chi transformation
- $HW(\cdot)$ is the Hamming Weight function

The Hamming Distance model is justified by the architecture of the target implementation, where the leakage is more correlated with internal bit transitions than absolute values, because:

- State registers are updated synchronously
- Transitions occur at clock edges
- No masking or countermeasures are present

5.4 Global Leakage Computation

In order to improve signal-to-noise ratio, leakage is aggregated over all 25 lanes of the Keccak state:

```
hd_leakage = np.zeros(len(msg), dtype=np.float32)

for x in range(5):
    for y in range(5):
        diff = state_initial[x, y] ^ chi_state_target[x, y]
        lane_hd = np.array([bin(int(v)).count("1") for v in diff])
        hd_leakage += lane_hd
```

This aggregation reflects the fact that multiple registers switch simultaneously during a Keccak round.

5.5 Usage of the Model

The leakage model developed in this chapter connects the physical power traces to the internal mathematical states of the cryptographic core. By mapping register transitions with the global Hamming Distance, it was possible to create a theoretical metric that estimates the chip's power fluctuations.

The next chapter will statistically evaluate this model against the recorded side-channel traces using Pearson correlation analysis. This step will be useful to isolate the Points of Interest (POIs) where the Keccak implementation leaks the most information.

6 Correlation Power Analysis (CPA)

This section presents the application of Correlation Power Analysis (CPA) to the Keccak hardware implementation under study.

6.1 Leakage Model Used in CPA

The leakage model adopted for CPA is based on the Hamming Distance between consecutive states of the Keccak permutation:

$$L(t) = HW(S_{old}(t) \oplus S_{new}(t))$$

where S_{old} and S_{new} represent the internal state before and after the Chi transformation.

In practice, the leakage is aggregated across all 25 lanes of the state to improve signal-to-noise ratio.

6.2 Correlation Metric

CPA relies on the Pearson correlation coefficient, defined as:

$$\rho_{T,L} = \frac{\text{cov}(T, L)}{\sigma_T \sigma_L}$$

where:

- T represents the power consumption traces
- L represents the hypothetical leakage model

A higher absolute correlation indicates a stronger dependency between leakage and measured traces.

6.3 Point of Interest Selection

Each sample point in the trace is evaluated independently.

The correlation curve along the traces is:

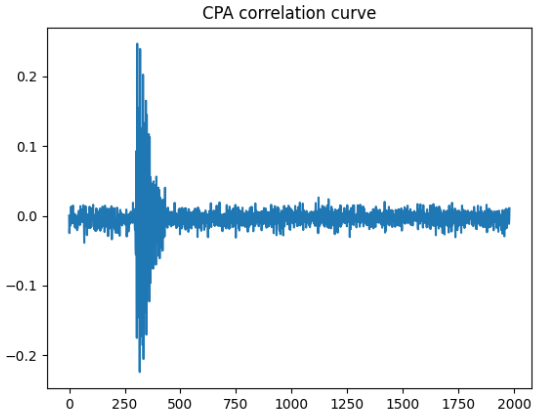

Figure 7: Correlation Power Analysis Curve

The Points of Interest (POIs) are selected as the time samples with highest absolute correlation:

```
top_k_POI = np.argsort(np.abs(corrs))[-50:]
```

The best 50 POIs identified correspond to the most leakage-relevant regions of the trace.

6.4 Experimental Results

The Best 50 POIs found are:

```
[350 301 379 420 352 337 366 300 392 403 340 407 309 324 353 390 325 354 377
 310 339 311 302 364 356 338 341 351 323 326 362 358 360 328 313 336 343 349
 308 315 345 347 321 304 330 332 334 317 319 306]
```

As visible in Figure 7, the selected POIs are highly clustered within a narrow temporal window.

This temporal proximity is a significant finding that validates the physical soundness of the leakage model. From a hardware perspective, it indicates that the side-channel leakage is not a mathematical artifact or random noise, but a direct consequence of a specific physical operation. This tight concentration of high-correlation points corresponds precisely to the clock cycles where the target 64-bit lane of the internal state is being actively processed.

By restricting the input of the CNN exclusively to this dense region of interest, it is possible to discard irrelevant and noisy time samples. This significantly reduces the network's input dimensionality, preventing the deep learning model from overfitting on environmental noise and drastically accelerating the training process.

The maximum observed correlation value is:

$$\rho_{max} = 0.247$$

Although a correlation coefficient of this magnitude is considered moderate from a statistical point of view, it represents a solid and distinct leakage signal in the context of hardware Side-Channel Analysis. Due to environmental noise, which heavily affects real power traces, and the simultaneous activity of parallel hardware components, a correlation close to 25% confirms the target intermediate state leakage. Furthermore, the fact that this correlation is maintained across multiple adjacent samples within the cluster is consistent with the high sampling rate of the acquisition setup (10 samples per clock cycle), ensuring that the leakage is distinct and well-localized.

6.5 Transition to Machine Learning Approaches

CPA successfully identifies informative regions of the trace, but its predictive power remains limited, mainly because of the non-linear leakage behavior introduced by the the Chi transformation, and for the presence of noise.

As a result, CPA alone is insufficient to reliably recover internal state bits.

The limitations of CPA motivate the use of more advanced techniques capable of modeling non-linear relationships between traces and leakage.

7 Deep Learning Profiling Attack

This section describes the application of a Convolutional Neural Network (CNN) to perform a profiling side-channel attack against the Keccak implementation.

The objective of the network is to predict the 64-bit output of a selected Keccak lane, specifically:

$$\chi[3, 0]$$

7.1 Dataset Preparation

The original dataset, consisting of power traces and corresponding inputs, was divided into two independent subsets:

1. The profiling set contains 9900 traces and is used for model training and validation.
2. A separate attack set containing 100 traces is reserved exclusively for attack evaluation and is never used during the training process.

The inputs are preprocessed as follows:

- Selection of 50 Points of Interest (POIs)
- Standardization using zero mean and unit variance
- Reshaping for CNN input

```
X_poi = traces[:, top_k_POI]
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_cnn = X_train_scaled.reshape((..., ..., 1))
```

The output labels correspond to the bitwise decomposition of the target lane:

```
labels[:, i] = (recovery_lane >> i) & 1
```

7.2 CNN Architecture

The proposed architecture is a 1D CNN designed for side-channel trace classification.

```
final_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_cnn.shape[1], 1)),
    tf.keras.layers.Conv1D(filters=best_hparams['filters'], kernel_size=3,
    ↪  activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.AveragePooling1D(pool_size=2),
```

```
    tf.keras.layers.Conv1D(filters=best_hparams['filters'] * 2, kernel_size=3,
    ↪  activation='relu', padding='same'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.AveragePooling1D(pool_size=2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu',
    ↪  kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    tf.keras.layers.Dropout(best_hparams['dropout']),
    tf.keras.layers.Dense(64, activation='sigmoid')
])
```

Where `best_hparams` are calculated using an hyperparameter optimization method, explained in Subsection 7.4.

The final layer outputs 64 independent probabilities, each corresponding to a bit of the target state lane.

7.3 Training Procedure

The model is trained using binary cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{64} \left(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right)$$

Optimization is performed using Adam with early stopping based on validation loss.

7.4 Hyperparameter Optimization

A grid search is performed over the following parameters:

- Number of filters: 16, 32
- Learning rate: 1e-3, 5e-4
- Dropout rate: 0.3, 0.5

The best configuration obtained is:

Filters	Learning rate	Dropout
16	0.001	0.5

Table 2: CNN configuration

7.5 Experimental Results

The CNN achieves the following performance on the test set:

- Average attack bit recovery accuracy: 75.53%
- Best attack trace: 53/64 recovered bits (82.81%)

- Worst attack trace: 43/64 recovered bits (67.19%)

The profiling performances are shown in the classification report:

	Precision	Recall	F1-score
Class 0	77%	96%	85%
Class 1	57%	15%	24%

Table 3: Profiling results

As observed in Table 3, the model training shows a significant performance disparity between the target classes, with a high recall for class 0 but a remarkably low recall for class 1. This evident bias toward predicting the majority class is a well-known challenge in deep learning-based SCA, particularly when dealing with multi-label configurations, such as the Flexible Bit Model.

Two main reasons for this behavior are: first, the physical leakage associated with a bit being set to 1 might be masked by environmental noise or misalignment in the recorded traces, making the features of class 1 harder for the network to extract compared to the baseline state of class 0, and second, standard optimization functions like Binary Cross-Entropy (BCE) treat all misclassifications equally. When a dataset presents an inherent or functional imbalance during model training, the loss function tends to converge to a local minimum that favors the majority class to minimize the overall loss quickly.

Several techniques could theoretically mitigate this issue:

- Class-Weighted Loss Functions: It is possible to adjust the BCE loss by assigning a higher weight coefficient to class 1 errors, making the gradients penalize false negatives more severely.
- Focal Loss: Using Focal Loss instead of BCE can down-weight the loss assigned to easy-to-classify examples (class 0) and focus the network's learning capacity on hard examples (class 1).
- Data Resampling: Employing oversampling techniques on the minority class instances or synthetic trace generation to balance the training distribution.

These solutions were not implemented within the scope of this project due to specific methodological and practical constraints. Tuning class weights or implementing Focal Loss introduces additional hyperparameter dimensions, which would exponentially increase the computational search space. Furthermore, artificial data resampling risks introducing synthetic artifacts into the side-channel traces, potentially biasing the CNN toward non-physical leakage characteristics and compromising the soundness of the physical evaluation. Ultimately, despite this imbalance, the current baseline results and the obtained bit recovery accuracy were already sufficient and acceptable to fulfill the primary objective of providing soft information for a future SASCA framework.

A profiling attack was performed on the attack dataset containing 100 traces.

For each attack trace, the CNN produces a vector of 64 probabilities corresponding to the targeted lane bits.

The final prediction is obtained using a threshold of 0.5:

```
pred = (attack_prob > 0.5).astype(int)
```

The success rate is computed as:

$$SR = \frac{\text{correct bits}}{64}$$

The average bit recovery accuracy over the attack set is 75.53%, and the best attack trace achieves a recovery of 53 out of 64 bits (82.81%), while the worst attack trace recovers 43 out of 64 bits (67.19%).

The results demonstrate that deep learning represents a significant improvement over CPA in terms of bit recovery capability.

This improvement is attributed to the ability to model non-linear leakage dependencies and to the robustness of the model to noise and temporal misalignment.

However, some limitations remain, such as the class imbalance which affects the prediction stability and the variation of the performances across different bit positions.

7.6 From Profiling to SASCA

The probabilistic outputs of the CNN can be interpreted as soft information about internal state bits. This makes them suitable for integration into a Soft Analytical Side-Channel Attack (SASCA) framework.

In this framework, the CNN outputs can be combined with algebraic constraints of Keccak in a factor graph, enabling iterative belief propagation for full state recovery.

The CNN developed in this work outputs a probability estimate for each of the 64 target bits.

Rather than producing only hard binary decisions, these probabilities preserve information about the confidence of the prediction.

In a SASCA framework, these probabilities could be combined with the algebraic constraints imposed by the Keccak round function through factor-graph based inference techniques such as Belief Propagation.

The implementation of this extension falls outside the scope of the present project and constitutes its next stage.

8 Results

This section presents a comparative evaluation of the two main side-channel attack methodologies analyzed in this project: Correlation Power Analysis (CPA) and Convolutional Neural Network (CNN)-based profiling attacks.

The goal is to assess their effectiveness in recovering internal state information from the Keccak hardware implementation and to highlight their respective strengths and limitations.

8.1 Evaluation Metrics

The performance of both approaches is evaluated using the following metrics:

- **Correlation Coefficient (ρ)**: measures linear dependency between leakage model and traces
- **Bit-wise Accuracy**: percentage of correctly predicted bits
- **Average Bit Recovery Accuracy**: success probability of bit recovery trace by trace

For the CNN model, bit-wise prediction is performed using a threshold of 0.5 on the output probabilities.

8.2 CPA Results

CPA results show a maximum observed correlation coefficient of:

$$\rho_{max} = 0.2469$$

While this value indicates a moderate linear dependency from a strictly statistical perspective, it represents a very distinct and solid leakage signal within the practical constraints of hardware SCA. This correlation confirms that the target intermediate state reliably leaks information despite environmental noise and the simultaneous activity of other parallel components on the chip.

Despite successfully identifying the leakage regions based on the POIs selection, CPA alone remains limited by its linear assumptions and does not directly allow a reliable bit recovery of the target state.

8.3 CNN Results

The Convolutional Neural Network achieves significantly better performance in terms of bit recovery.

The main results are summarized as follows:

- Average bit recovery accuracy on the attack set: 75.53%
- Best attack trace: 53 correctly recovered bits out of 64
- Worst attack trace: 43 correctly recovered bits out of 64

These results demonstrate that the CNN is capable of learning complex non-linear relationships between power traces and internal state bits.

8.4 Discussion

The experimental results clearly indicate that deep learning-based profiling attacks outperform classical CPA in the considered scenario.

This improvement is mainly due to the ability of CNNs to capture non-linear leakage relationships, learn temporal dependencies in raw traces, and reduce reliance on explicit leakage modeling.

Moreover, CPA is limited by its implicit assumption of linear correlation between leakage and power consumption.

8.5 Security Implications

From a cryptographic security perspective, the results underline that unprotected implementations of Keccak are vulnerable to profiling attacks.

The relatively high bit recovery rate achieved by the CNN suggests that a significant portion of the internal state can be inferred from a small number of traces.

Although the recovered information corresponds to an internal state lane rather than a secret cryptographic key or the input message, the results demonstrate that meaningful internal information can be extracted from power measurements.

This confirms the vulnerability of unprotected hardware implementations to advanced profiling attacks, reinforcing the importance of implementing dedicated side-channel countermeasures.

9 Conclusion

This project investigated the side-channel vulnerability of an unprotected hardware implementation of the Keccak-f[1600] permutation. The main objective was to analyze the leakage behavior of the design and evaluate the effectiveness of both classical and modern attack techniques.

The work combined three main components:

- A detailed analysis of the VHDL implementation of Keccak
- A leakage modeling framework based on Hamming Distance
- Two attack methodologies: Correlation Power Analysis (CPA) and Deep Learning-based profiling attacks

Additionally, a conceptual framework for a Soft Analytical Side-Channel Attack (SASCA) was introduced as a future extension.

9.1 Key Contributions

The main contributions of this project can be summarized as follows:

- Reverse engineering of the target Keccak implementation.
- Development of an emulator of first-round Keccak computations.
- Construction of a Hamming Distance leakage model.
- Identification of leakage locations through CPA-based feature selection.
- Design and evaluation of a CNN-based profiling attack.
- Definition of a probabilistic output format suitable for future SASCA integration.

9.2 Experimental Findings

The experimental results demonstrate that the target implementation exhibits measurable side-channel leakage.

The CPA analysis confirmed the presence of exploitable leakage, but was not effective in recovering the internal state bits, because of Chi function's non-linearity.

The profiling attack achieved significantly better results, using a CNN and confirming the superiority of deep-learning approaches in modeling non-linear leakage behavior.

9.3 Limitations

Despite the promising results, several limitations remain:

- Generalization to different devices or acquisition setups is not guaranteed
- Class imbalance affects prediction quality for certain bit positions
- The SASCA framework has not been fully implemented and remains theoretical

9.4 Future Work

There are several directions for future research that can be identified:

- Full implementation of the SASCA framework using factor graphs and belief propagation
- Extension of the attack to multiple rounds of Keccak-f[1600]
- Evaluation of countermeasures such as masking and hiding techniques

9.5 Final Remarks

This project developed the profiling stage required for a future SASCA attack against the hardware Keccak-f[1600] implementation.

By combining reverse engineering, leakage modeling, CPA-based feature selection, and deep-learning profiling, the proposed framework extracts probabilistic information about internal Keccak state variables. These probabilistic outputs build the foundation for the future SASCA.

References

[1] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Keccak team - figures. Figures of the Keccak-f implementation. URL: `https://keccak.team/figures.html`.

[2] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. The sponge and duplex constructions. Keccak sponge construction figure. URL: `https://keccak.team/sponge_duplex.html`.

[3] Patrik Dobias. Keccak vhdl implementation, 2024. Source code provided.

[4] Patrik Dobias, Lukas Malina, and Jan Hajny. Efficient unified architecture for post-quantum cryptography: combining dilithium and kyber. *PeerJ Computer Science*, 2025.

[5] Patrik Dobias, Azade Rezaeezade, Lejla Batina, Lukasz Chmielewski, and Lukas Malina. Sok: Reassessing side-channel vulnerabilities and countermeasures in pqc implementations, 2025.

[6] Anna Guinet, Georg Land, Ioan Gabriel Bucur, and Tim Güneysu. A tale of snakes and horses: Amplifying correlation power analysis on quadratic maps. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024.

[7] Matthias J. Kannwischer, Peter Pessl, and Robert Primas. Single-trace attacks on keccak. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020, 2020.

[8] Julien Maillard, Thomas Susen Hiscock, Maxime Lecomte, and Christophe Clavier. Simulating sasca on keccak: Security implications for post-quantum cryptographic schemes. In *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)*, pages 518–527, 2024.

[9] NIST. Fips pub 202: Sha-3 standard: Permutation-based hash and extendable-output functions. Technical report, National Institute of Standards and Technology, August 2015.

[10] George Provelengios, Nicolas Sklavos, Paris Kitsos, and Christos Koulamas. Fpga-based design approaches of keccak hash function. In *2012 15th Euromicro Conference on Digital System Design (DSD)*. IEEE, 2012.

[11] Niels Samwel and Joan Daemen. Dpa on hardware implementations of ascon and keyak. In *Proceedings of the Computing Frontiers Conference (CF'17)*, 2017.

[12] Shih-Chun You and Markus G. Kuhn. Single-trace fragment template attack on a 32-bit implementation of keccak. In *International Conference on Smart Card Research and Advanced Applications (CARDIS 2021)*. Springer, 2021.

Appendices

A Code Availability

The complete source code developed for this thesis is publicly available.

Repository:

`https://github.com/Kelfx/Keccak_SCA`

The repository includes the software emulator of the first Keccak round, leakage-model generation scripts, CPA analysis tools, CNN training pipelines, and evaluation scripts.